

Final Project Report

Introduction to Geometric Deep Learning

Anya Sukachev and Sergiy Horef

October 16, 2024

1 Task 2: Graph Classification

1.1 Method and Architecture choice

We have started by trying the GAT model we have seen in class.

In order to use this model for graph classification, we have added a readout function (function that takes embeddings of all nodes and creates a unified embedding based on some aggregation function), and a fully connected layer which takes the result produced by the readout function and returns a list of probabilities of each class.

We have created a variable implementation, such that we could dynamically define the number of GNN layers, number of attention head, readout aggregation function, and the hidden dimension of the fully connected network.

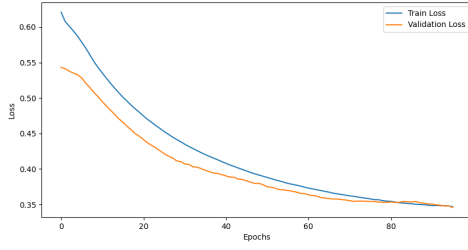
Because from our initial runs (with some randomly chosen parameters) we were already able to get to a validation accuracy of 100%, we did not feel the need to try a different model altogether, and instead used wandb to make hyperparameter optimization.

All variables we have used in the optimization can be found in q2_wandb.py, but in total we have looked at 1800 different combinations of parameters. Later, we chose only the parameter sets which resulted in validation accuracy of 100% and all of those we sorted by the minimal validation loss, and overall stability.

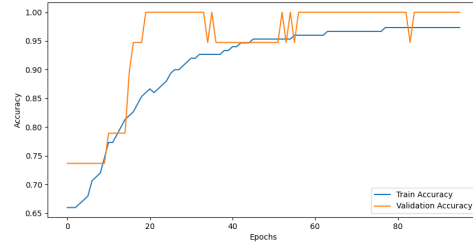
Our final model has the following results on the training and validation sets after 96 epochs of training:

Train loss: **0.3468**, Train accuracy: **0.9733**

Validation loss: **0.3457**, Validation accuracy: **1.0000**



(a) Loss on Train and Validation



(b) Accuracy on Train and Validation

Figure 1: Plots of the performance of our final model for Task 2

1.2 Discussion of Results

We have a very small dataset, which includes only 150 training points, 19 validation and 19 testing points.

It is relatively easy to get a high accuracy on 19 points in total, and the complexity of the model is expressive enough to achieve an almost perfect accuracy on train. On the contrary, we can see small jumps on the graph of the validation accuracy, which result from the fact that even a single mistake is equivalent to around 5.23% in accuracy.

One thing which we would like to point out in addition to everything that we have said before - the training and validation sets have an unequal number of 0s and 1s, with ratio of 1s to 0s as approximately 2:1. Therefore we have manually set the weight in the loss function we have used to give twice more weight to the 0 label.